# C Variables

Understanding how to declare, define, and use variables in C programming

# What is a Variable?

A variable is nothing but a name given to a storage area that our programs can manipulate. Each variable in C has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable.

## Variable Characteristics

The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C is case-sensitive.

### Basic Types

Based on the basic types explained in the previous chapter, there will be the following basic variable types:

## Char

Typically a single octet (one byte)

## Int

The most natural size of integer for the machine

## Float

A single-precision floating point value


## Double

A double-precision floating point value


## Void

Represents the absence of type

> 💡 **Note**
>
> C programming language also allows to define various other types of variables, which we will cover in subsequent chapters like Enumeration, Pointer, Array, Structure, Union, etc. For this chapter, let us study only basic variable types.

# Rules for Declaring Variable Name

| Rule | Description |
| --- | --- |
| **Length** | Variable name may be a combination of alphabet, digits, or underscores and its length should not exceed eight characters. |
| **First Character** | First character must be an alphabet. |
| **Special Characters** | No commas or blank spaces are allowed in variable name. |
| **Underscore** | Among the special symbols, only underscore can be used in variable name. |

## ✅ Valid Variable Names

```
int emp_age;
int item_4;
```

⚠️ **Important**

Following these rules ensures your variable names are valid and consistent with C programming standards.

# Variable Definition in C

A variable definition tells the compiler where and how much storage to create for the variable. A variable definition specifies a data type and contains a list of one or more variables of that type as follows:

```
Type Variable_list;
```

## 📝 Multiple Variable Declaration

```
int i, j, k;
char c, ch;
float f, salary;
double d;
```

💡 **Initialization**

Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression:

```
Type variable_name = value;
```

## 🔧 Declaration with Initialization

```
extern int d = 3, f = 5; // declaration of d and f
int d = 3, f = 5; // definition and initializing d and f
```

💡 **Uninitialized Variables**

For definition without an initializer: variables with static storage duration are implicitly initialized with NULL (all bytes have the value 0); the initial value of all other variables is undefined.



# Variable Declaration in C

A variable declaration provides assurance to the compiler that there exists a variable with the given type and name so that the **compiler can proceed for further compilation without requiring the complete detail about the variable.**

All the variables must be declared before their use. Declaration does two things:

### Name Specification

It tells the compiler what the variable name is.

### Type Specification

It specifies the type of data, the variable will hold.

Declaration

Type_specifier list_of_variables;

↓

Definition

Type variable_name = value;

```
// Variable declaration:
extern int a, b;
extern int c;
extern float f;
```

```
// Variable definition:
int a, b;
```

```
int c;
float f;
```

value of c : 30
value of f : 23.333334

> 💡 **Declaration vs Definition**
>
> Though you can declare a variable multiple times in your C program, it can be defined only once in a file, a function, or a block of code.

## Declaration

```
extern int a, b;
```

✅ Only declares variables

VS

VS

## Definition

```
int a, b;
```

✅ Declares and defines variables

# Function Declaration

The same concept applies on function declaration where you provide a function name at the time of its declaration and its actual definition can be given anywhere else.

```
// function declaration
int func();

// function call
int i = func();

// function definition
int func() {
  return 0;
}
```

0

💡 **Function Declaration**

Function declarations are useful when you are using multiple files and you define your variable in one file which will be available at the time of linking the program.



# Conclusion

As a result, data types are essential in the C programming language because they define the kinds of information that variables can hold. They provide the data's size and format, enabling the compiler to allot memory and carry out the necessary actions.

## Key Points to Remember

### Data Types

Define the kinds of information that variables can hold

### Memory Management

Specify the size and format of data

### Type Selection

Choose the right type for the data

🙏 Thank you for your attention! 👋